LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 RELASI ANTAR KELAS

NANDA MAYLA ADIASTI

103132400003

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Konsep Dasar Diagram Kelas (Class Diagram)

Diagram kelas (class diagram) adalah jenis diagram struktur statis dalam UML (Unified Modeling Language) yang menggambarkan struktur sistem dengan memodelkan kelas-kelas, atribut, operasi (method), dan hubungan antar objek di dalam sistem. Sebuah kelas digambarkan sebagai kotak yang terbagi menjadi tiga bagian utama:

a. Baris Pertama: Menyatakan nama kelas (Class Name).
b. Baris Kedua: Menyatakan daftar atribut beserta tipe datanya.
c. Baris Ketiga: Menyatakan daftar method atau operasi yang dimiliki oleh kelas tersebut.

Dalam pemrograman berorientasi objek, visibilitas atau access modifier menentukan tingkat aksesibilitas suatu atribut atau method, yang disimbolkan dengan:

a. +Public (dapat diakses dari luar maupun dalam kelas).
b. - : Private (hanya dapat diakses di dalam kelas itu sendiri).
c. # : Protected (dapat diakses dalam kelas yang sama dan sub-kelasnya).

Hubungan Antar Kelas (Class Relationships)

Dalam Object-Oriented Programming (OOP), kelas-kelas yang saling berinteraksi memiliki bentuk hubungan tertentu sesuai dengan skenario nyata yang dimodelkan. Hubungan antar kelas ini secara umum dibagi menjadi empat jenis utama, yaitu asosiasi, agregasi, komposisi, dan generalisasi.

Asosiasi (Association)

Asosiasi adalah hubungan struktural antara dua kelas terpisah di mana satu kelas menggunakan atau berinteraksi dengan kelas lainnya.

1. Karakteristik: Hubungan ini bersifat longgar (loosely coupled). Objek dari kedua kelas dapat berdiri sendiri dan diciptakan secara terpisah. Salah satu kelas menggunakan method dari kelas lain melalui parameter method, tanpa menjadikan objek tersebut sebagai atribut permanen (field utama).
2. Contoh Implementasi: Kelas Dosen berasosiasi dengan kelas Mahasiswa, di mana dosen dapat memproses nilai mahasiswa dengan mengirimkan objek mahasiswa sebagai parameter tanpa memilikinya secara fisik.

Agregasi (Aggregation)

Agregasi adalah bentuk khusus dari asosiasi yang menyatakan hubungan bagian dan keseluruhan (part-of) dengan ikatan yang tidak terlalu mutlak.

1. Karakteristik: Kelas wadah (whole) menampung kumpulan objek bagian (part) menggunakan array atau struktur data lainnya. Meskipun objek bagian menjadi bagian dari objek wadah, siklus hidup keduanya independen; artinya, jika objek wadah dihapus, objek bagian tetap dapat berdiri sendiri.

2. Contoh Implementasi: Kelas Jurusan menampung array objek Mahasiswa (DaftarMahasiswa). Objek mahasiswa dibuat terlebih dahulu di luar, kemudian dimasukkan ke dalam jurusan melalui method tertentu.

Komposisi (Composition)

Komposisi adalah bentuk hubungan agregasi yang jauh lebih kuat dan ketat (strongly coupled).

1. Karakteristik: Objek bagian (part) merupakan bagian internal yang mutlak dari objek keseluruhan (whole). Objek bagian tidak dapat berdiri sendiri dan tidak pernah dibuat secara terpisah di luar kelas utamanya. Siklus hidup objek bagian terikat penuh pada objek utama; jika objek utama dimusnahkan, maka seluruh objek bagian di dalamnya akan ikut musnah.
2. Contoh Implementasi: Sebuah Komputer tersusun atas komponen internal seperti CPU, Monitor, dan Mouse, di mana objek komponen tersebut di-instansiasi secara langsung di dalam constructor kelas komputer.

B. Guided

1. Asosiasi

[Mahasiswa.java]

```
package guided1.Asosiasi;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNim(String nim) {
        this.nim = nim;
    }

    public String getNim() {
        return this.nim;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }

    public String getNama() {
        return this.nama;
    }

    public void setNilai(int nilai) {
        this.nilai = nilai;
    }

    public int getNilai() {
        return this.nilai;
    }
}
```

[Dosen.java]

```
package guided1.Asosiasi;

public class Dosen {

    private String kodeDosen;
    private String namaDosen;
    private int nilai;

    //Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    //Getter
    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
    s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
    return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("103132400003");
        m.setNama("Mayla Cantik");
        Dosen d = new Dosen();
        d.giveScore(m, 100);
        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul05 ---
Nim :103132400003
Nama :Mayla Cantik
Nilai :100
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.040 s
Finished at: 2026-10-18T10:40:06+07:00
------------------------------------------------------------------------
```

2. Agregasi

[Mahasiswa.java]

```
package guided2.Agregasi;

public class Mahasiswa {
   private String NIM, Nama;

   public Mahasiswa(String NIM, String Nama) {
      this.NIM = NIM;
      this.Nama = Nama;
   }

   //Getter
   public String GetNIM() {
      return (NIM);
   }

   public String GetNama() {
      return (Nama);
   }

   //Setter
   public void SetNim(String NIM){
      this.NIM = NIM;
   }

   public void SetNama(String Nama){
      this.Nama = Nama;
   }
}
```

[Jurusan.java]

```
package guided2.Agregasi;

public class Jurusan {

    private static int JumlahMahasiswa = 0;
    private String KodeJurusan, NamaJurusan;
    private Mahasiswa[] DaftarMahasiswa = new Mahasiswa[32];

    // Constructor
    public Jurusan(String kodeJurusan, String namaJurusan) {
        this.KodeJurusan = kodeJurusan;
        this.NamaJurusan = namaJurusan;
    }

    // Behaviour
    public void AddMahasiswa (Mahasiswa m) {
        this.DaftarMahasiswa[JumlahMahasiswa] = m;
        JumlahMahasiswa++;
    }

    public void DisplayMahasiswa() {
        System.out.println("Kode Jurusan : " + this.KodeJurusan);
        System.out.println("Nama Jurusan : " + this.NamaJurusan);

        System.out.println("Daftar Mahasiswa :");

        for (int i = 0; i < JumlahMahasiswa; i++) {
            System.out.println((i + 1) + ". " + DaftarMahasiswa[i].GetNIM() +
" " + DaftarMahasiswa[i].GetNama());
        }
    }

    // Getter
    public String GetKodeJurusan() {
        return KodeJurusan;
    }

    public String GetNamaJurusan() {
        return NamaJurusan;
    }

    public static int GetJumlahMahasiswa() {
        return JumlahMahasiswa;
    }
```

```
    public Mahasiswa[] GetDaftarMahasiswa() {
        return DaftarMahasiswa;
    }

    // Setter
    public void SetKodeJurusan(String KodeJurusan) {
        this.KodeJurusan = KodeJurusan;
    }

    public void SetNamaJurusan(String NamaJurusan) {
        this.NamaJurusan = NamaJurusan;
    }
}
```

[Main.java]

```
package guided2.Agregasi;

public class Main {
    public static void main(String[] args) {
        runAsosiasi();
    }

    public static void runAgregasi() {}

    public static void runAsosiasi() {
        Mahasiswa m1 = new Mahasiswa("103132400003", "Mayla Cantik");
        Mahasiswa m2 = new Mahasiswa("103132400100", "Gaih Pargoy");

        Jurusan prodi = new Jurusan("SD", "Sains Data");

        prodi.AddMahasiswa(m1);
        prodi.AddMahasiswa(m2);

        prodi.DisplayMahasiswa();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul05 ---
Kode Jurusan : SD
Nama Jurusan : Sains Data
Daftar Mahasiswa :
1. 103132400003 Mayla Cantik
2. 103132400100 Gaih Pargoy
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.309 s
Finished at: 2026-10-18T10:42:32+07:00
------------------------------------------------------------------------
```

3. Komposisi

[Processor.java]

```
package guided3.Komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

[Monitor.java]

```
package guided3.Komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

[Mouse.java]

```
package guided3.Komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

[Komputer.java]

```
package guided3.Komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String
brandMonitor, String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }

}
```

[Main.java]

```
package guided3.Komposisi;

import guided2.Agregasi.*;
import guided3.Komposisi.*;

public class Main {
    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }


    public static void runGuided1(){}

    public static void runGuided2(){
        Mahasiswa m1 = new Mahasiswa("103132400003", "Mayla cantik");
        Mahasiswa m2 = new Mahasiswa("103132400100", "Galih ganteng");

        Jurusan prodi = new Jurusan("SD","Sains Data");

        prodi.AddMahasiswa(m1);
        prodi.AddMahasiswa(m2);
        prodi.DisplayMahasiswa();

    }

    public static void runGuided3(){
        Komputer pc = new Komputer("Intel Core i5",3200,"LG 24
Inch","Logitech","Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul05 ---
Kode Jurusan : SD
Nama Jurusan : Sains Data
Daftar Mahasiswa :
1. 103132400003 Mayla cantik
2. 103132400100 Galih ganteng
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.268 s
Finished at: 2026-10-18T10:45:47+07:00
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

   Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

   Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (Main).

   Kebutuhan Tugas :

   1. Membuat dua class: RincianHarga dan ItemPesanan. (Komposisi)
   2. Membuat class Pesanan (Agregasi)
   3. Membuat class Kasir (Asosiasi)
   4. Membuat class Main untuk pengujian

   Output yang diHarapkan :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

[Kasir.java]

```
package unguided.Cafe;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] items = pesanan.getDaftarItem();
        int jumlah = pesanan.getJumlahItem();

        for (int i = 0; i < jumlah; i++) {
            totalSubtotal += items[i].getRincian().getHargaDasar();
            totalPajak += items[i].getRincian().getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + jumlah);
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

[ItemPesanan.java]

```
package unguided.Cafe;

public class ItemPesanan {
   private String namaItem;
   private RincianHarga rincian; // Objek internal (Komposisi)

   public ItemPesanan(String namaItem, double hargaDasar) {
      this.namaItem = namaItem;
      this.rincian = new RincianHarga(hargaDasar); // Di-instansiasi langsung di
constructor
   }

   public String getNamaItem() {
      return namaItem;
   }

   public RincianHarga getRincian() {
      return rincian;
   }
}
```

[Pesanan.java]

```
package unguided.Cafe;

public class Pesanan {
   private int nomorMeja;
   private ItemPesanan[] daftarItem; // Array penampung (Agregasi)
   private int jumlahItem;

   public Pesanan(int nomorMeja, int kapasitas) {
      this.nomorMeja = nomorMeja;
      this.daftarItem = new ItemPesanan[kapasitas];
      this.jumlahItem = 0;
   }

   public void tambahItem(ItemPesanan item) {
      if (jumlahItem < daftarItem.length) {
         daftarItem[jumlahItem] = item;
         jumlahItem++;
      }
   }

   public void tampilkanPesanan() {
      System.out.println("=== DAFTAR PESANAN CAFE ===");
      System.out.println("Nomor Meja: " + this.nomorMeja);
      for (int i = 0; i < jumlahItem; i++) {
         ItemPesanan item = daftarItem[i];
```

```
                System.out.println((i + 1) + ". " + item.getNamaItem() + " - Rp " +
        item.getRincian().getHargaTotal() +
                        " (Harga: " + item.getRincian().getHargaDasar() + ", Pajak: " +
        item.getRincian().getPajak() + ")");
            }
            System.out.println();
        }

        public int getNomorMeja() {
            return nomorMeja;
        }

        public ItemPesanan[] getDaftarItem() {
            return daftarItem;
        }

        public int getJumlahItem() {
            return jumlahItem;
        }
}
```

[RincianHarga.java]

```
        package unguided.Cafe;

        public class RincianHarga {
            private double hargaDasar;
            private double pajak;

            public RincianHarga(double hargaDasar) {
                this.hargaDasar = hargaDasar;
                this.pajak = hargaDasar * 0.1; // Pajak 10%
            }

            public double getHargaDasar() {
                return hargaDasar;
            }

            public double getPajak() {
                return pajak;
            }

            public double getHargaTotal() {
                return this.hargaDasar + this.pajak;
            }
}
```

[Main.java]

```
package unguided.Cafe;

public class Main {
    public static void main(String[] args) {
        // 1. Buat 1 objek Kasir
        Kasir kasir = new Kasir("Budi");

        // 2. Buat 3 objek ItemPesanan secara independen (Agregasi)
        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        // 3. Buat 1 objek Pesanan untuk Meja 4 (kapasitas 5 item)
        Pesanan pesananMeja4 = new Pesanan(4, 5);

        // 4. Masukkan ketiga objek ItemPesanan ke dalam objek Pesanan
        pesananMeja4.tambahItem(item1);
        pesananMeja4.tambahItem(item2);
        pesananMeja4.tambahItem(item3);

        // 5. Tampilkan rincian pesanan
        pesananMeja4.tampilkanPesanan();

        // 6. Jalankan pemrosesan pembayaran oleh kasir (Asosiasi)
        kasir.prosesPembayaran(pesananMeja4);
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul05 ---
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
================================
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  3.662 s
Finished at: 2026-10-18T09:38:00+07:00
------------------------------------------------------------------------
```

Penjelasan :

A. Kasir.java

   Kelas Kasir merepresentasikan petugas kasir yang bertugas memproses pembayaran transaksi di cafe. Kelas ini menerapkan hubungan asosiasi dengan kelas Pesanan, di mana kelas Kasir tidak menyimpan objek Pesanan sebagai atribut atau field permanen, melainkan menerimanya melalui parameter pada method prosesPembayaran(Pesanan pesanan). Di dalam method ini, kasir mengambil daftar item dan jumlah item dari objek pesanan, lalu melakukan iterasi menggunakan perulangan untuk mengakumulasi total harga dasar (subtotal) dan total pajak dari seluruh menu yang dibeli. Setelah perhitungan total keseluruhan selesai, kasir mencetak struk pembayaran yang memuat nama kasir, nomor meja, jumlah item, total subtotal, total pajak, hingga total bayar akhir.

B. ItemPesanan.java

   Kelas ItemPesanan berfungsi untuk merepresentasikan setiap menu makanan atau minuman yang dipesan oleh pelanggan. Kelas ini memiliki atribut berupa namaItem (nama menu) dan objek rincian yang bertipe RincianHarga. Hubungan antara ItemPesanan dan RincianHarga dirancang menggunakan konsep komposisi, di mana objek RincianHarga di-instansiasi secara langsung di dalam constructor ItemPesanan menggunakan kata kunci new. Hal ini memastikan bahwa objek rincian harga terikat secara mutlak dan tidak pernah dibuat secara terpisah di luar kelas ini. Selain itu, kelas ini menyediakan method getter untuk mengambil nama item dan objek rincian harga.

C. Pesanan.java

   Kelas Pesanan berperan sebagai wadah yang menampung kumpulan item pesanan untuk nomor meja tertentu. Kelas ini menerapkan hubungan agregasi, di mana ia memiliki array daftarItem bertipe ItemPesanan[] beserta variabel jumlahItem untuk menandai banyaknya item yang sudah masuk. Objek-objek ItemPesanan dibuat secara independen di luar kelas ini, lalu dimasukkan satu per satu ke dalam wadah array melalui method tambahItem(ItemPesanan item). Selain itu, kelas ini dilengkapi dengan method tampilkanPesanan() yang melakukan perulangan untuk mencetak daftar menu yang dipesan beserta rincian harga dasar, pajak, dan total harga per item secara terstruktur.

D. RincianHarga.java

   Kelas RincianHarga bertugas untuk mengelola rincian finansial dari suatu item pesanan, yang meliputi pencatatan harga dasar dan perhitungan besaran pajak. Atribut di dalamnya mencakup hargaDasar dan pajak. Melalui constructor-nya, kelas ini secara otomatis menghitung nilai pajak sebesar 10% dari harga dasar yang dikirimkan. Kelas ini juga menyediakan method getHargaTotal() untuk mengembalikan nilai akumulasi akhir dari penjumlahan harga dasar dan pajak.

E. Main.java

Kelas Main berfungsi sebagai titik awal eksekusi program (entry point) untuk menjalankan dan menguji seluruh relasi kelas yang telah dibuat. Di dalam method main, alur program dimulai dengan menginisialisasi satu objek Kasir bernama "Budi", lalu membuat tiga objek ItemPesanan secara independen beserta harga dasarnya (Agregasi). Selanjutnya, program membentuk objek Pesanan untuk Meja 4 dengan kapasitas tertentu, lalu memasukkan ketiga item pesanan ke dalamnya menggunakan method tambahItem(). Setelah itu, program memanggil method tampilkanPesanan() untuk menampilkan rincian daftar menu meja 4, dan ditutup dengan memanggil method prosesPembayaran(pesananMeja4) dari objek kasir untuk menyelesaikan transaksi serta mencetak struk pembayaran akhir (Asosiasi).

D. Kesimpulan

1. Praktikan telah memahami dan mampu menerapkan konsep dasar Pemrograman Berorientasi Objek (PBO) dalam merancang struktur program berbasis relasi antar kelas, yang meliputi asosiasi, agregasi, dan komposisi.
2. Hubungan komposisi berhasil diimplementasikan antara kelas ItemPesanan dan RincianHarga, di mana objek bagian di-instansiasi secara langsung di dalam constructor kelas utama sehingga siklus hidupnya terikat secara mutlak.
3. Hubungan agregasi diterapkan pada kelas Pesanan yang menampung objek ItemPesanan secara independen menggunakan array standar, memungkinkan objek bagian tetap ada meskipun wadahnya dimanipulasi.
4. Hubungan asosiasi dibuktikan melalui interaksi antara kelas Kasir dan Pesanan, di mana objek kasir memproses data pesanan melalui parameter method tanpa menjadikan objek pesanan sebagai atribut permanen.